Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральное государственное бюджетное образовательное учреждение высшегообразования
«Сибирский государственный университет телекоммуникаций и информатики»
Уральский технический институт связи и информатики (филиал) в г. Екатеринбурге
ЦК ИТ и АСУ кафедры ИСТ

Отчет
по учебной практике МДК 02.02 «Инструментальные средства разработки программного обеспечения»
на тему «Разработка программного модуля для учета заявок на ремонт бытовой техники»

студента 3 курса 381 группы

Фамилия Удилова

Имя, отчество Екатерина Вячеславовна

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы и программирование»

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение 4
1 Анализ предметной области 5
1.1 Краткая спецификация разрабатываемого модуля 5
2 Проектирование базы данных 10
2.1 Описание базы данных 10
2.2 ER – диаграмма 12
3 Этап разработки базы данных 13
3.1 Демонстрация основных запросов на создание БД 13
3.2 Демонстрация схемы разработанной базы данных 14
4 Разработка дизайна сервиса 16
4.1 Алгоритм работы и интерфейс приложения 16
4.3 Система контроля качества и генерация QR-кода 17
4.4 Расчет статистики 18
5 Разработка страниц сервиса 19
5.1 Форма авторизации 19
5.2 Главное окно (Интерфейс Оператора/Менеджера) 19
5.3 Форма добавления и редактирования заявки 20
5.4 Рабочее место Мастера и Менеджера по качеству 21
5.5 Генерация QR-кода для отзыва 21
5.6 Отчетность (Статистика) 22
6 Тестирование 23
6.1 Тестирование работы приложения 23
7 Описание практической части дипломного проекта 25
7.1 Обоснование выбора инструментальных средств разработки 25
8 Разработка приложения 28
8.1 Разработка структур веб-приложения 28
Заключение 38
Приложение А 40
Приложение Б 41

Введение

В современных условиях эффективная работа сервисного центра по ремонту бытовой техники невозможна без автоматизации процессов учета и управления заявками. Рост количества клиентов и разнообразие обслуживаемого оборудования (холодильники, стиральные машины, плиты и др.) в ООО «БытСервис» привели к необходимости создания специализированного программного модуля.

Актуальность темы обусловлена тем, что переход от ручного ведения журналов и разрозненных таблиц к единой реляционной базе данных позволяет минимизировать риск потери заявок, сократить время на их обработку и обеспечить прозрачность работы мастеров для руководства.

Целью данной работы является разработка и внедрение программного модуля для автоматизации учета заявок в ООО «БытСервис», который позволит объединить данные о клиентах, оборудовании и ходе ремонтных работ в единую информационную систему.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проанализировать предметную область и существующие бизнес-процессы сервисного центра.
2. Спроектировать структуру базы данных на основе имеющихся таблиц учета, обеспечив целостность и нормализацию данных.
3. Разработать программный модуль с разграничением прав доступа (оператор, мастер, менеджер по качеству).
4. Реализовать функционал для отслеживания статусов ремонта, фиксации расхода запчастей и ведения комментариев.
5. Внедрить систему контроля качества на основе генерации QR-кодов для получения обратной связи от клиентов.
6. Протестировать разработанное ПО на соответствие требованиям технического задания.

Объектом исследования является процесс управления сервисным обслуживанием в ООО «БытСервис». Предметом исследования является разработка программного инструментария для автоматизации учета и мониторинга исполнения заявок на ремонт.

Практическая значимость работы заключается в создании готового решения, которое повышает скорость взаимодействия между сотрудниками и улучшает качество сервиса за счет оперативного информирования клиентов и контроля сроков выполнения работ.

1 Анализ предметной области

1.1 Краткая спецификация разрабатываемого модуля

Предметной областью данного проекта является деятельность сервисного центра по ремонту бытовой техники.

Основной задачей сервисного центра является прием заявок от клиентов, выполнение ремонтных работ и выдача отремонтированной техники. Для выполнения этих задач необходимо вести учет клиентов, заявок, сотрудников и этапов выполнения работ.

В работе сервисного центра участвуют несколько категорий пользователей:

- клиент - обращается в сервисный центр с заявкой на ремонт техники;
- оператор - принимает заявку, регистрирует ее в системе и указывает основные данные о неисправности;
- мастер - выполняет ремонт техники и фиксирует этапы выполнения работ;
- менеджер - контролирует выполнение заявок и распределяет задачи между сотрудниками;
- менеджер по качеству - анализирует качество обслуживания и контролирует соблюдение сроков выполнения работ.

Основными данными, которые необходимо хранить в системе, являются:

- информация о клиентах;
- данные о сотрудниках;
- сведения о заявках;
- комментарии к заявкам;
- статус выполнения работ.

Автоматизация данного процесса позволяет:

- ускорить регистрацию заявок;
- контролировать сроки выполнения ремонта;
- распределять нагрузку между сотрудниками;
- хранить историю работы с клиентами.

Таким образом, разработка информационной системы учета заявок является актуальной задачей для повышения эффективности работы сервисного центра (Рисунок 1.1).

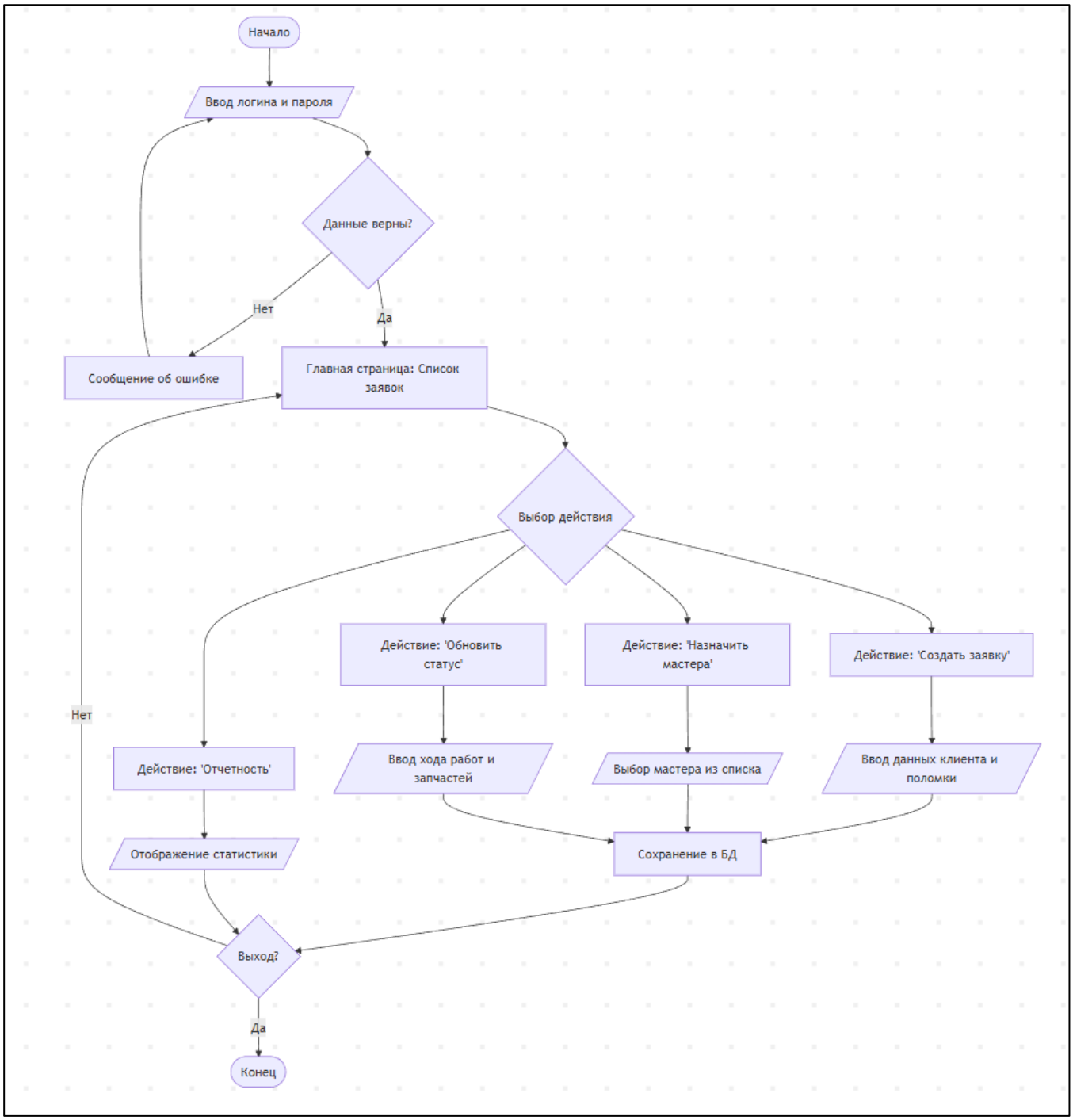

Рисунок 1.1 – Блок-схема основного алгоритма

Блок-схема функции расчёта количества заявок (Рисунок 1.2).

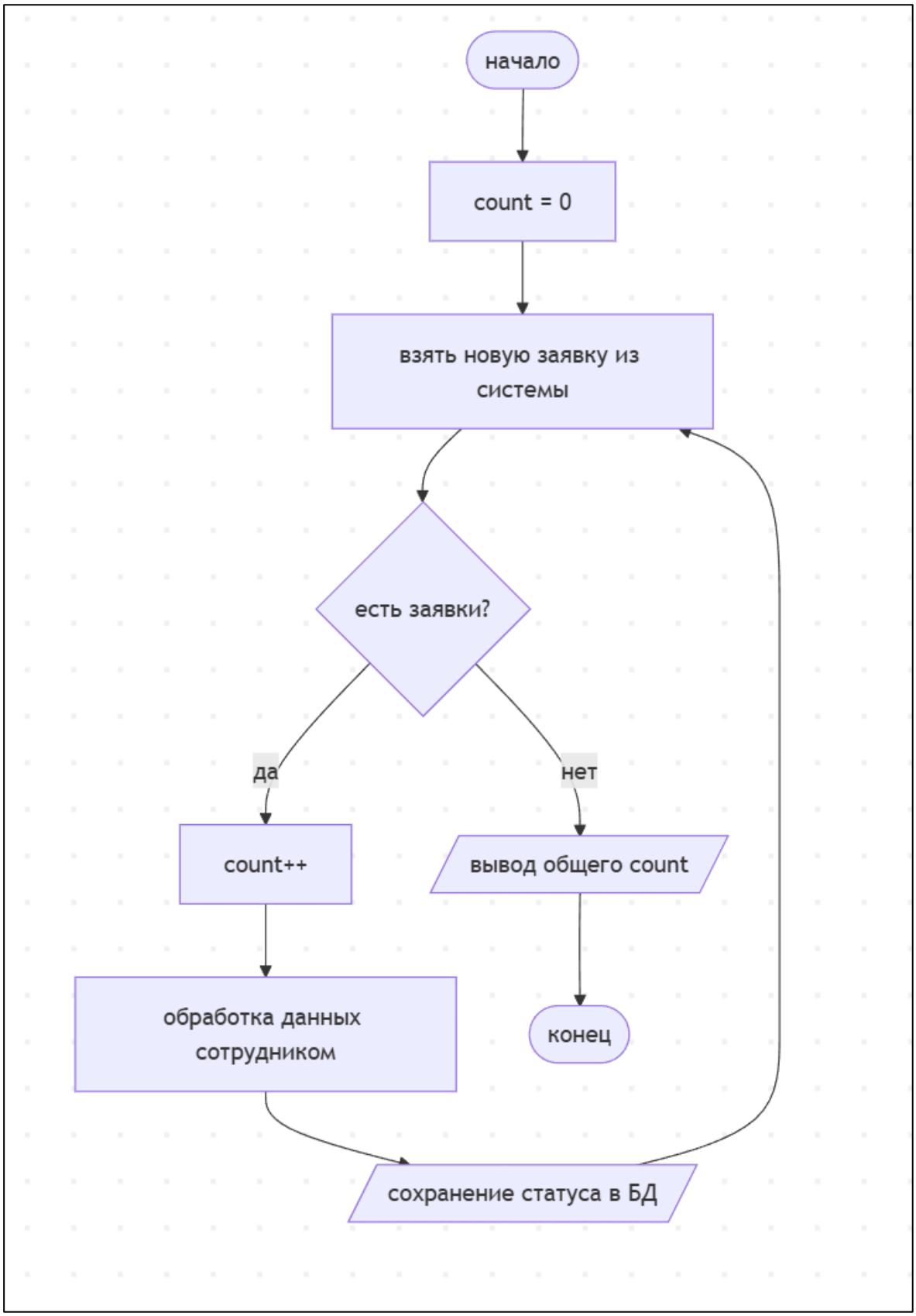

Рисунок 1.2 - Блок-схема функции расчёта количества заявок

1.2 Функциональные и нефункциональные требования

Функциональные требования

В ходе разработки программного модуля были реализованы следующие функциональные возможности:

1. Управление записями о ремонте:

- Регистрация новых заявок в базе данных с фиксацией полного набора параметров: уникальный номер, дата постановки на учет, категория бытовой техники, конкретная модель, подробное описание неисправности, контактные данные клиента (ФИО, номер телефона) и текущий статус.
- Редактирование существующих заявок, включая корректировку описания проблемы, изменение этапа выполнения работ и назначение/смену ответственного исполнителя.

2. Мониторинг и навигация:

- Формирование и визуализация общего списка заявок для оперативного контроля.
- Многокритериальная система поиска: как по уникальному идентификатору (номеру) заявки, так и по косвенным параметрам (модель техники, статус и т.д.).
- Инструменты отслеживания жизненного цикла заявки в реальном времени.

3. Взаимодействие сотрудников:

- Механизм назначения мастеров (исполнителей) на конкретные задачи.
- Функционал для мастеров по ведению рабочих журналов: добавление комментариев к заявке, фиксация использованных запчастей и расходных материалов.

4. Аналитическая отчетность:

- Автоматический расчет количественных показателей (число успешно закрытых заявок).
- Вычисление временных метрик (средняя продолжительность выполнения ремонта).
- Формирование статистических данных по типам неисправностей для оптимизации работы склада запчастей.

Нефункциональные требования

Разработанная система отвечает следующим критериям качества:

1. Совместимость: Обеспечена полная поддержка работы приложения в среде ОС семейства Windows.
2. Эргономичность: Интерфейс спроектирован с упором на простоту и интуитивность; система оснащена информативными уведомлениями и подсказками для пользователя.

3. Производительность: Оптимизированы запросы к базе данных, что гарантирует высокую скорость обработки информации и минимальное время отклика интерфейса.

4. Безопасность: Реализована многоуровневая система авторизации с разграничением прав доступа в зависимости от роли пользователя (Клиент, Оператор, Мастер, Менеджер по качеству), что исключает несанкционированный доступ к конфиденциальным данным.

2 Проектирование базы данных

2.1 Описание базы данных

Структура базы данных была предоставлена в рамках задания в виде набора таблиц. Данные таблицы содержали информацию о пользователях системы, клиентах, заявках и комментариях.

В процессе выполнения практики была выполнена адаптация и перенос структуры данных в реляционную базу данных SQLite. Для этого были созданы соответствующие таблицы и настроены связи между ними с использованием внешних ключей.

Для хранения данных были разработаны таблицы, структура которых представлена в таблицах 1-6.

Таблица 1 - Структура таблицы «Roles»

Имя поля	Описание	Тип данных
roleID	Идентификатор роли	[PK] integer
roleName	Название роли	string

Таблица 2 – Структура таблицы «Users»

Имя поля	Описание	Тип данных
userID	Уникальный идентификатор пользователя	[PK] integer
fio	Фамилия имя и отчество пользователя	string
phone	Номер телефона	string
login	Логин	string
password	Пароль	string
roleID	Ссылка на таблицу ролей	[FK] integer

Таблица 3 - Структура таблицы «Clients»

Имя поля	Описание	Тип данных
clientID	Идентификатор клиента	[PK] integer
fio	Фамилия имя и отчество	string

Продолжение таблицы 3

phone	Номер телефона	string

Таблица 4 - Структура таблицы «Requests»

Имя поля	Описание	Тип данных
requestID	идентификатор заявки	[PK] integer
startDate	дата создания заявки	date
homeTechType	вид устройства	string
homeTechModel	модель устройства	string
problemDescription	описание проблемы	string
completionDate	дата фактического завершения работ	date
repairParts	перечень заказанных запчастей	string
statusID	ссылка на статус	[FK] integer
clientID	ссылка на клиента	[FK] integer
masterID	ссылка на мастера	[FK] integer

Таблица 5 - Структура таблицы «Statuses»

Имя поля	Описание	Тип данных
statusID	идентификатор статуса	[PK] integer
statusName	статус	string

Таблица 6 - Структура таблицы «Comments»

Имя поля	Описание	Тип данных
commentID	идентификатор комментария	[PK] integer
message	Текст сообщения о ходе ремонта	string
masterID	ссылка на мастера	[FK] integer
requestID	ссылка на заявку	[FK] integer

2.2 ER – диаграмма

На основе выделенных таблиц была построена схема связей «сущность-связь». Основная связь реализуется между таблицей users и requests. Одна запись в users (клиент) может иметь множество связанных записей в requests (связь 1:M). Аналогично мастер связан с заявками, которые он выполняет.

Переход к такой структуре позволил ООО «БытСервис» отойти от избыточности: теперь ФИО клиента и его телефон хранятся в базе один раз, даже если он обращался в сервис десять раз с разной техникой. (Рисунок 2.2)

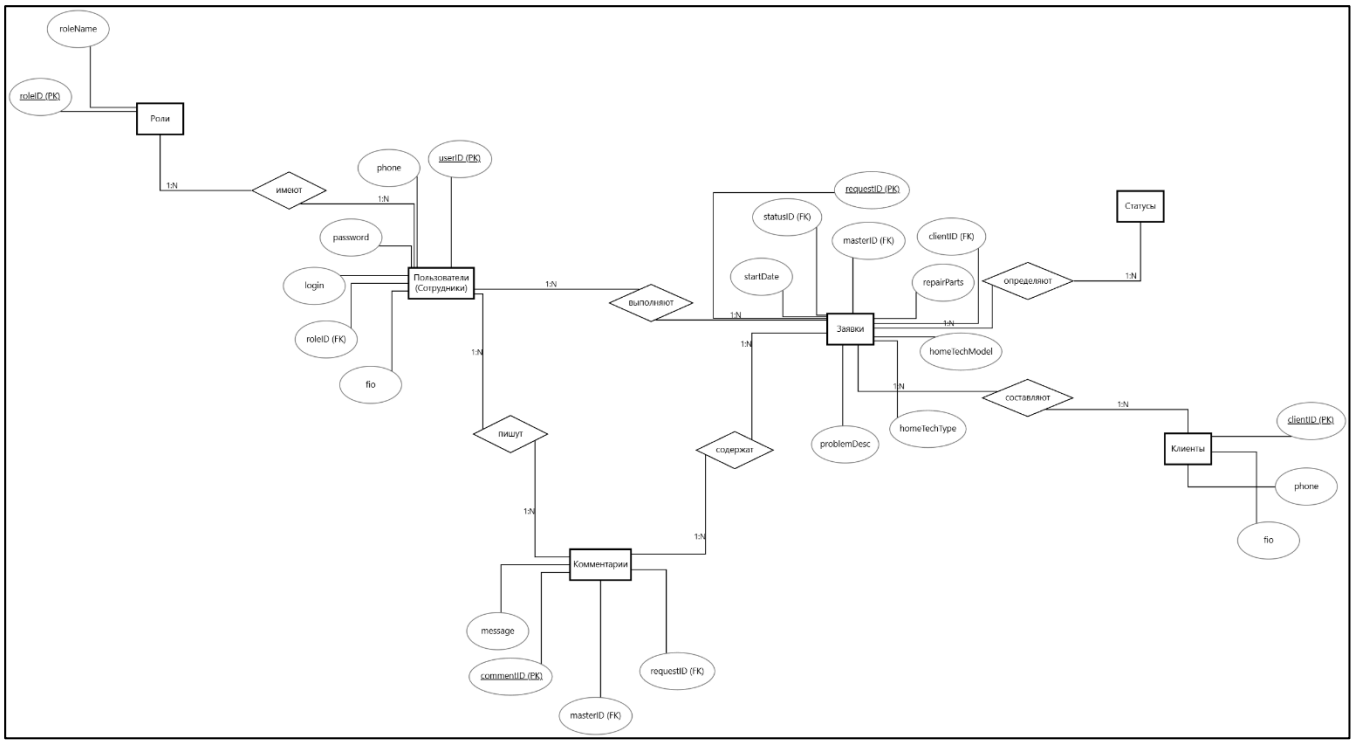

Рисунок 2.2 – ER-диаграмма

3 Этап разработки базы данных

3.1 Демонстрация основных запросов на создание БД

В данном разделе представлена программная реализация базы данных. Поскольку работа велась по принципу переноса данных из готовых таблиц учета в реляционную СУБД SQLite, создание структуры и наполнение базы осуществлялось с помощью SQL-запросов, интегрированных в код приложения.

Листинг кода 3.1 - Создание таблиц-справочников (Роли и Статусы)

```
CREATE TABLE Roles (
    roleID INTEGER PRIMARY KEY AUTOINCREMENT,
    roleName VARCHAR(255) NOT NULL
);

CREATE TABLE Statuses (
    statusID INTEGER PRIMARY KEY AUTOINCREMENT,
    statusName VARCHAR(255) NOT NULL
);
```

Листинг кода 3.2 - Создание таблиц пользователей и клиентов

```
CREATE TABLE Users (
    userID INTEGER PRIMARY KEY AUTOINCREMENT,
    fio VARCHAR(255) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    login VARCHAR(50) UNIQUE,
    password VARCHAR(50),
    roleID INTEGER,
    FOREIGN KEY (roleID) REFERENCES Roles(roleID)
);

CREATE TABLE Clients (
    clientID INTEGER PRIMARY KEY AUTOINCREMENT,
    fio VARCHAR(255) NOT NULL,
    phone VARCHAR(20) NOT NULL
);
```

Листинг кода 3.3 - Создание основной таблицы заявок

```
CREATE TABLE Requests (
    requestID INTEGER PRIMARY KEY AUTOINCREMENT,
    startDate DATE NOT NULL,
    homeTechType VARCHAR(255),
    homeTechModel VARCHAR(255),
    problemDescription TEXT,
    completionDate DATE,
    repairParts TEXT,
    statusID INTEGER,
    clientID INTEGER,
    masterID INTEGER,
    FOREIGN KEY (statusID) REFERENCES Statuses(statusID),
    FOREIGN KEY (clientID) REFERENCES Clients(clientID),
    FOREIGN KEY (masterID) REFERENCES Users(userID)
);
```

Листинг кода 3.4 - Создание таблицы комментариев

```
CREATE TABLE Comments (
    commentID INTEGER PRIMARY KEY AUTOINCREMENT,
    message TEXT NOT NULL,
    masterID INTEGER,
    requestID INTEGER,
    FOREIGN KEY (masterID) REFERENCES Users(userID),
    FOREIGN KEY (requestID) REFERENCES Requests(requestID)
);
```

Эти SQL-запросы создают базу данных и необходимые таблицы. Они включают создание таблиц, определение первичных ключей и установление внешних ключей для обеспечения целостности данных и связей между таблицами. Эти запросы служат основой для дальнейшего развития и использования базы данных в системе.

3.2 Демонстрация схемы разработанной базы данных

На этом этапе будет представлена схема разработанной базы данных, которая включает в себя все таблицы, их атрибуты и отношения между ними. Схема поможет визуализировать структуру базы данных и понять, как данные взаимодействуют друг с другом. (Рисунок 3.2)

Рисунок 3.2 – Разработаная схема БД

Схема разработанной базы данных включает в себя все необходимые таблицы, их атрибуты и отношения между ними. Эта схема служит основой для дальнейшего развития и использования базы данных в системе. Визуализация схемы с помощью ERD-диаграммы помогает лучше понять структуру базы данных и взаимодействие данных в системе.

4 Разработка дизайна сервиса

4.1 Алгоритм работы и интерфейс приложения

В данном разделе описывается разработка клиентской части приложения на языке Python с использованием библиотеки Tkinter для создания графического интерфейса. Программа реализует взаимодействие с 6-ю таблицами базы данных, обеспечивая выполнение бизнес-процессов ООО «БытСервис».

Приложение построено по модульному принципу, где каждый экран соответствует правам доступа пользователя. Основное окно представляет собой рабочую область с таблицей заявок, данными из которой подгружаются из связанного справочника Statuses.

Основные функции интерфейса:

1. Авторизация. Проверка пары логин/пароль в таблице Users и определение roleID для фильтрации доступного функционала.
2. просмотр и поиск. Отображение списка из таблицы Requests с возможностью фильтрации по модели техники или номеру заявки.
3. редактирование. Возможность изменения полей problemDescription и statusID.

4.2 Реализация функционала по ролям

Логика работы программного модуля построена на разграничении прав доступа. При входе в систему приложение проверяет roleID пользователя и предоставляет соответствующий интерфейс:

1. Заказчик (Клиент):

– Подача заявки: самостоятельное заполнение формы с указанием типа техники (homeTechType), модели (homeTechModel) и описания проблемы. При этом в таблице Requests автоматически фиксируется его clientID.

– Мониторинг: просмотр списка своих заявок и их текущих статусов (подгружаются из таблицы Statuses).

– Редактирование: возможность изменить описание проблемы, если заявка еще не принята в работу.

– Обратная связь: после перевода заявки в статус «Готова к выдаче», клиенту открывается доступ к QR-коду для перехода на форму оценки качества.

2. Оператор:

– Регистрация: прием звонков и внесение данных о новых клиентах в таблицу Clients.

– Управление очередью: создание заявок и назначение первичного статуса «Новая заявка».

– Поиск: быстрая фильтрация всех заявок по номеру телефона клиента или модели устройства.

3. Мастер:

– Работа с заказом: мастер видит только те записи, где его userID совпадает с masterID в таблице заявок.

– Отчетность: добавление комментариев в таблицу Comments о ходе ремонта (например, «Проведена диагностика», «Ожидаю запчасти»).

– Запчасти: внесение данных в поле repairParts для учета материалов.

– Менеджер по качеству:

– Контроль сроков: если ремонт затягивается, менеджер может изменить completionDate после согласования с клиентом.

– Консультация: просмотр таблицы Comments для помощи мастерам в сложных случаях.

– Назначение: возможность привлечь другого мастера к выполнению заявки (изменение masterID).

4.3 Система контроля качества и генерация QR-кода

В соответствии с дополнением к ТЗ, в модуль был интегрирован функционал для оценки работы сервисного центра. При завершении ремонта (статус «Готова к выдаче»), система предлагает сгенерировать QR-код.

Алгоритм генерации:

1. Используется библиотека qrcode.
2. В код зашивается ссылка на Google-форму
3. QR-код выводится на экран оператора или распечатывается для клиента.

Листинг 4.2 - Код генерации QR-кода:

```
      def show_feedback_qr():
    url =
"https://docs.google.com/forms/d/e/1FAIpQLSdhZcExx6LSIXxk0ub55mSu-
WIh23WYdGG9HY5EZhLDo7P8eA/viewform?usp=sf_link"
    qr = qrcode.QRCode(version=1, box_size=10, border=5)
    qr.add_data(url)
    qr.make(fit=True)
    img = qr.make_image(fill_color="black", back_color="white")
    top = tk.Toplevel()
    top.title("Оцените нашу работу")
    img_tk = ImageTk.PhotoImage(img)
    label = tk.Label(top, image=img_tk)
    label.image = img_tk
      label.pack(padx=20, pady=20)
    tk.Label(top, text="Отсканируйте код для отзыва").pack(pady=10)
```

4.4 Расчет статистики

Для менеджеров ООО «БытСервис» реализован блок аналитики, выполняющий SQL-запросы к таблице Requests.

Количество выполненных заявок: `SELECT COUNT(*) FROM Requests WHERE statusID = (SELECT statusID FROM Statuses WHERE statusName = 'Готова к выдаче')`.

5 Разработка страниц сервиса

5.1 Форма авторизации

При запуске приложения пользователю открывается окно авторизации. В зависимости от введенных данных (логин/пароль) из таблицы Users, система определяет роль пользователя и открывает соответствующий функционал. (Рисунок 5.1)

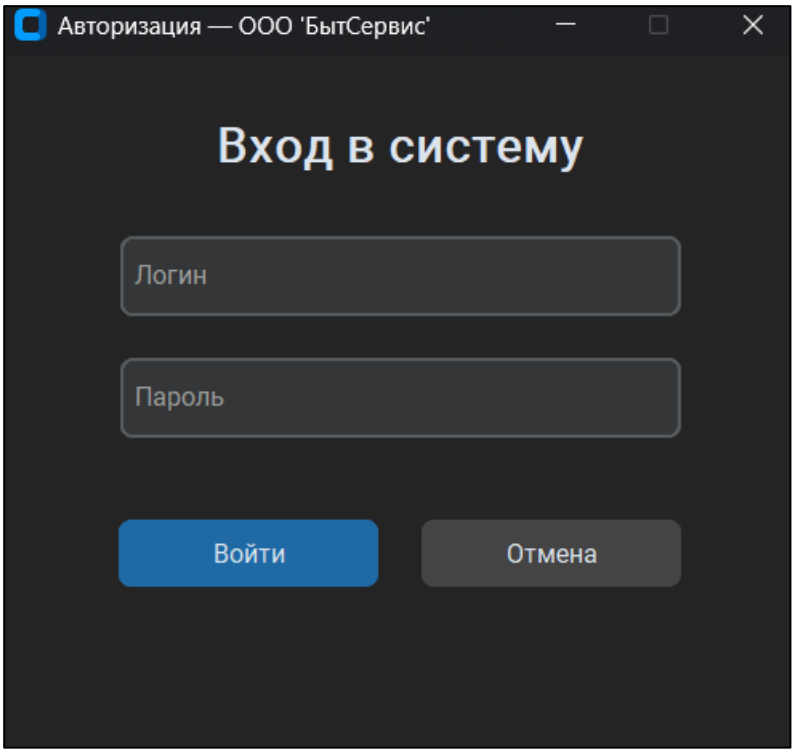

Рисунок 5.1 – Окно входа в систему

5.2 Главное окно (Интерфейс Оператора/Менеджера)

После входа открывается основная рабочая область. Центральным элементом является таблица (виджет Treeview), которая отображает объединенные данные из таблиц Requests, Clients и Statuses. (Рисунок 5.2)

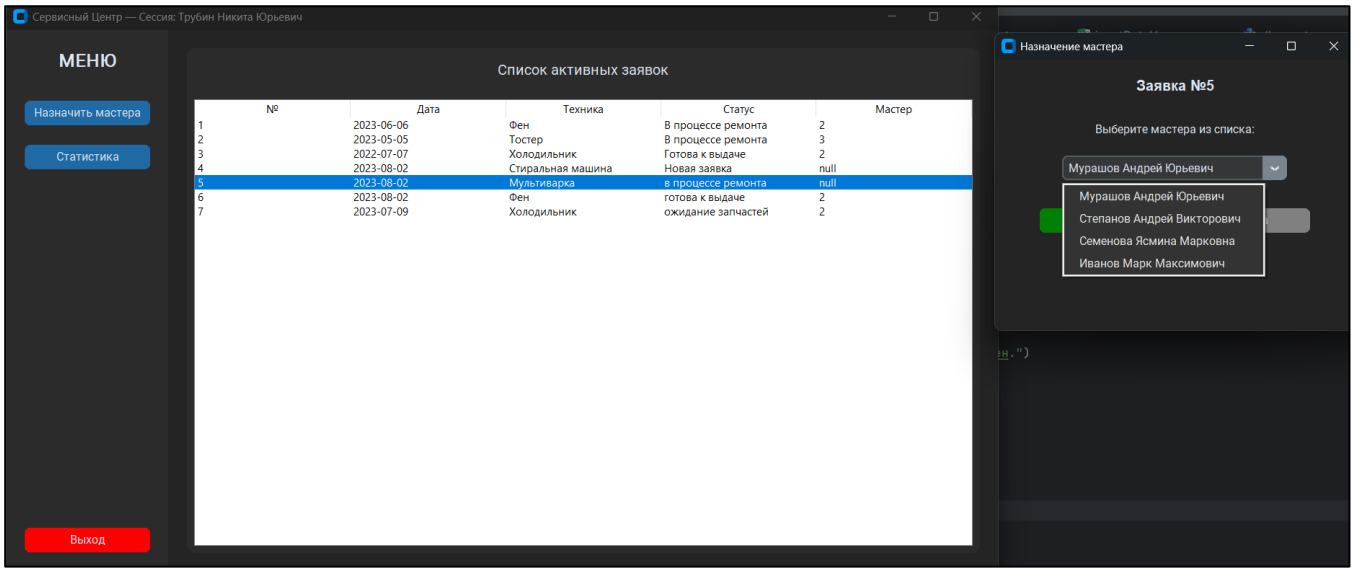

Рисунок 5.2 - Список всех заявок сервисного центра

5.3 Форма добавления и редактирования заявки

Оператор или Клиент могут добавить новую заявку. При этом данные распределяются: ФИО и телефон уходят в таблицу Clients, а параметры ремонта - в таблицу Requests. (Рисунок 5.3)

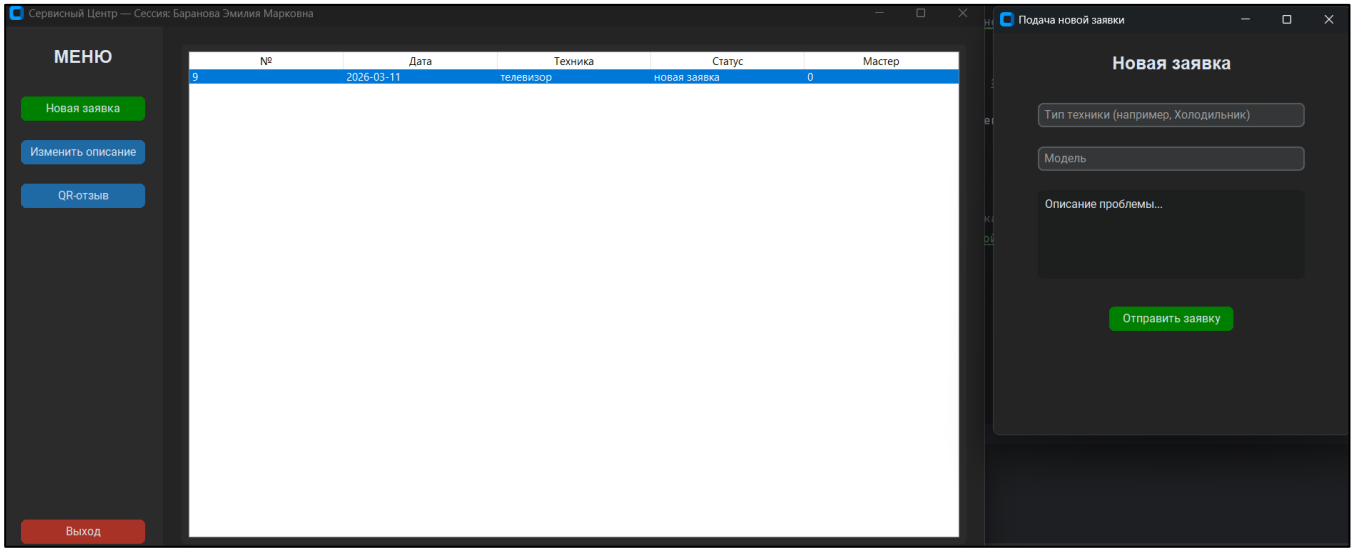

Рисунок 5.3 – Редактирование заявки в окне Оператора

5.4 Рабочее место Мастера и Менеджера по качеству

Мастер может открыть конкретную заявку, чтобы добавить комментарий (таблица Comments) или указать запчасти. Менеджер по качеству имеет кнопку «Продлить срок», которая меняет completionDate в базе. (Рисунок 5.4)

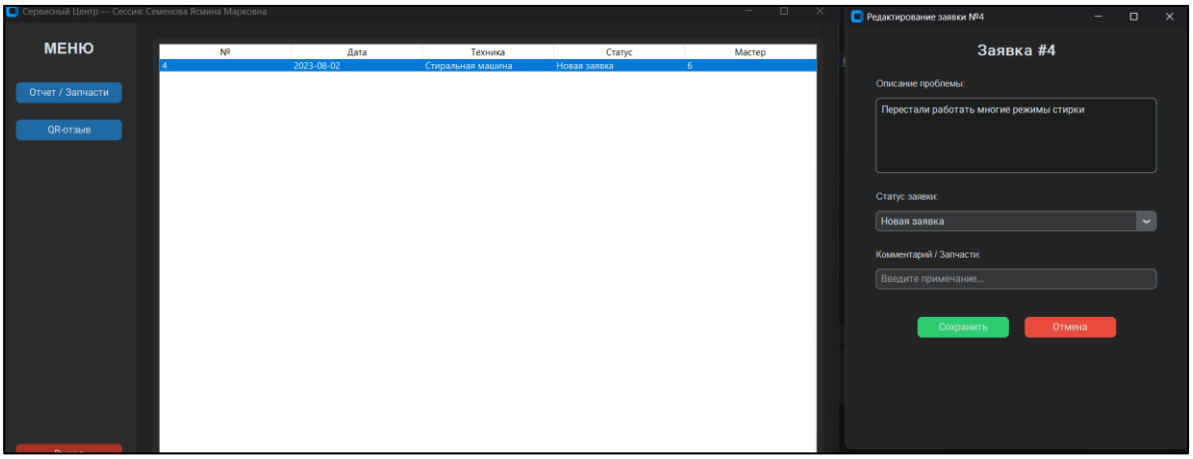

Рисунок 5.4 - Окно редактирования заявки мастером

5.5 Генерация QR-кода для отзыва

Согласно дополнению к ТЗ, при переводе заявки в статус «Готова к выдаче» (ID из таблицы Statuses), приложение выводит QR-код. (Рисунок 5.5)

Рисунок 5.5 - Всплывающее окно с QR-кодом

5.6 Отчетность (Статистика)

В приложении реализована вкладка «Отчеты», где программно вычисляется количество выполненных работ. (Рисунок 5.6)

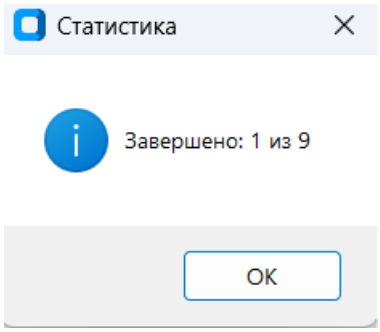

Рисунок 5.6 - Вывод статистики

6 Тестирование

6.1 Тестирование работы приложения

Завершающим этапом разработки является тестирование программного модуля. Цель данного этапа - подтвердить соответствие приложения функциональным и нефункциональным требованиям, указанным в ТЗ для ООО «БытСервис», а также проверить корректность взаимодействия всех 6-ти таблиц базы данных.

Тестирование проводилось методом «черного ящика». Проверялись основные сценарии использования системы:

1. авторизация. Проверка разграничения прав доступа по ролям.
2. операции с данными. Создание, поиск и редактирование заявок.
3. бизнес-логика. Смена статусов, добавление комментариев и генерация QR-кода.

Результаты проведенных испытаний представлены в Таблице 6.1.

Таблица 6.1 — Протокол тестирования программного модуля

№	Описание теста	Ожидаемый результат	Результат	Статус
1	Вход под ролью «Мастер»	Открывается интерфейс с доступом только к назначенным заявкам и таблице Comments.	Соответствует	Успешно
2	Попытка входа с неверным паролем	Система выдает предупреждение «Неверный логин или пароль».	Соответствует	Успешно
3	Создание заявки Клиентом	Новая запись успешно добавляется в таблицу Requests с текущей датой.	Соответствует	Успешно
4	Поиск заявки по модели техники	В таблице отображаются только те записи, которые содержат искомую строку (например, «LG»).	Соответствует	Успешно
5	Продление срока Менеджером	Поле `completionDate` в таблице Requests обновляется на новую дату.	Соответствует	Успешно
6	Смена статуса на «Готова к выдаче»	Программа генерирует окно с QR-кодом, содержащим ссылку на Google-форму.	Соответствует	Успешно

В ходе тестирования было подтверждено, что:

- доступ к данным строго ограничен. Клиент не может видеть заявки других пользователей;
- база данных SQLite корректно обрабатывает связи между 6-ю таблицами, предотвращая удаление связанных данных;
- приложение стабильно работает в среде ОС Windows, выполняя запросы к БД в реальном времени.

7 Описание практической части дипломного проекта

7.1 Обоснование выбора инструментальных средств разработки

Для реализации веб-приложения адаптивного обучения программированию необходимо выбрать инструментарий, обеспечивающий удобство разработки, расширяемость, кроссплатформенность и возможность дальнейшего сопровождения проекта.

В этом разделе проводится сравнение доступных решений и обосновывается выбор конкретных технологий, использованных при создании программного продукта.

В качестве языка серверной части выбран Python. Данный выбор обусловлен следующими преимуществами:

− низкий порог входа и скорость разработки. Python обладает лаконичным и понятным синтаксисом, что ускоряет процесс разработки и снижает вероятность ошибок;

− широкая экосистема библиотек. Для Python существует большое количество готовых решений для веб-разработки, в том числе Flask и Django, а также инструменты для аналитики и работы с данными; − кроссплатформенность. Python не зависит от конкретной операционной системы и может быть развернут на любом сервере;

− популярность в образовании. Python часто используется именно в обучающих проектах, что делает его логичным выбором для системы, предназначенной для школ программирования.

В качестве альтернатив могли рассматриваться JavaScript (Node.js), PHP и Java. Однако:

− Node.js требует большего уровня подготовки, а его высокая асинхронность усложняет начальную разработку;

− PHP часто используется для классических CMS, но менее удобен при построении современных микросервисных API;

− Java обладает высокой надежностью, но требует большего времени на разработку и сложнее в эксплуатации для небольших образовательных стартапов.

Таблица 7.1 – Сравнение языков программирования

Критерий	Python	JavaScript (Node.js)	PHP	Java
Скорость разработки	Высокая	Средняя, требует дополнительной подготовки	Средняя	Низкая
Простота синтаксиса	Очень высокая	Средняя	Средняя	Низкая

Продолжение таблицы 7.1

Наличие библиотек для веб-приложений	Большое количество (Flask, Django и др.)	Очень большое	Большое	Большое
Порог входа	Низкий	Средний	Средний	Высокий
Распространённость в обучении	Очень высокая	Средняя	Низкая	Низкая
Кроссплатформенность	Полная	Полная	Полная	Полная

Таким образом, Python оптимально подходит для целей системы: он сочетает скорость разработки, гибкость и доступность.

Для написания и отладки программного кода используется PyCharm, поскольку данная IDE:

- предоставляет встроенные средства проверки качества кода;
- позволяет быстро запускать сервер и тестировать API;
- включает управление зависимостями и виртуальными окружениями;
- значительно облегчает работу с крупными проектами.

Таблица 7.2 – Сравнение сред разработки

Критерий	PyCharm	VS Code	Sublime / Notepad++
Интеграция с Python	Полная, отладка и управление окружениями	Частичная через расширения	Нет
Наличие встроенных инструментов	Да (отладчик, работа с БД, тесты)	Зависит от плагинов	Минимальные
Удобство отладки	Высокое	Среднее	Низкое
Кривая обучения	Средняя	Средняя	Нулевая
Подходит для больших проектов	Да	Да	Нет
Вывод	Наиболее удобная среда для разработки на Python	Универсальна, но требует настройки	Не подходит для крупного проекта

PyCharm также превосходит стандартные текстовые редакторы (например, Notepad++, Sublime Text, VS Code) в плане интегрированных инструментов для Python-разработки, особенно при работе с фреймворками.

В качестве системы хранения данных применён облачный сервис Firebase, что объясняется:

- отсутствием необходимости развертывания собственной серверной базы данных;
- высокой скоростью интеграции с веб-клиентом;
- встроенной поддержкой авторизации и синхронизации данных;
- возможностью расширения функциональности без миграции на другой сервер.

Альтернативные варианты, такие как MySQL, PostgreSQL или SQLite, также могли быть использованы, но их использование потребовало бы:

- развертывания локального или удаленного сервера;
- настройки администрирования;
- дополнительных работ по безопасности.

Firebase позволяет сократить затраты времени и ресурсов и подходит для образовательной среды.

8 Разработка приложения

8.1 Разработка структур веб-приложения

Перед началом непосредственной реализации веб-приложения необходимо спроектировать его структуру и основные логические процессы. Для этого используются блок-схемы, которые позволяют наглядно представить последовательность действий, алгоритмы обработки данных и взаимодействие различных модулей системы.

Разработка блок-схем обеспечивает:

- понимание логики работы приложения;
- выявление потенциальных ошибок на этапе проектирования;
- упрощение этапа программирования и тестирования;
- основу для последующей документации и сопровождения системы.

Основные этапы работы приложения (Рисунок 8.1):

1. Главная страница:
   - отображение логотипа KiberOne;
   - три кнопки выбора роли: Администратор, Учитель, Ученик.
2. Администратор:
   - ввод логина и пароля;
   - попадание в кабинет администратора;
   - доступные функции: просмотр данных всех пользователей (учеников, учителей, администраторов), просмотр результатов тестов, добавление, редактирование, удаление пользователей, управление контентом и заданиями.
3. Учитель:
   - ввод логина и пароля учителя;
   - попадание в кабинет учителя;
   - доступные функции: просмотр списка своих учеников, просмотр результатов тестов учеников, добавление новых учеников, создание и добавление тестов.
4. Ученик (Студент):
   - ввод имени и фамилии;
   - выбор аватара для профиля;
   - попадание в кабинет ученика;
   - доступные функции: смена аватара;
   - блоки обучения: тренажер «Собери клавиатуру» – три уровня сложности, «Стамина онлайн» – тренажер скорости печати, работа с буквами, знаками, фразами и предложениями, адаптивный тест по уровням владения ПК

Рисунок 8.1 – Структура проекта

Разработанная блок-схема наглядно демонстрирует структуру веб-приложения KiberOne и последовательность действий пользователей в зависимости от их роли: администратора, учителя или ученика. Она позволяет понять логику взаимодействия всех компонентов системы, обеспечивая:

– чёткое разграничение функционала для разных типов пользователей;

– наглядное представление потоков данных и действий внутри приложения;

– основу для проектирования пользовательского интерфейса и дальнейшей реализации серверной логики;

– возможность планирования расширений и новых функций, таких как статистика по группе и будущие обучающие блоки для учеников.

Таким образом, блок-схема служит важным инструментом при

проектировании приложения, позволяя визуализировать архитектуру, определить ключевые модули и связи между ними, а также обеспечить согласованность всех элементов системы.

Для наглядного представления логики работы функции анализа результатов тестирования учеников была разработана блок-схема, отражающая последовательность выполнения основных этапов обработки данных (Рисунок 9.2).

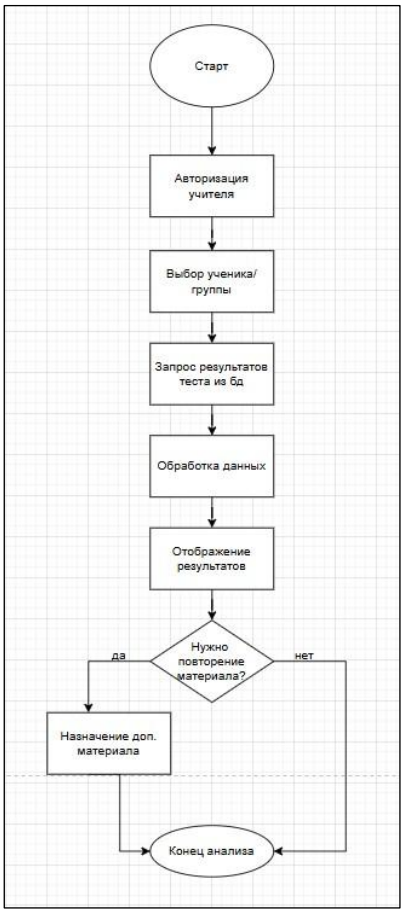

Рисунок 8.2 - Блок-схема анализа результатов тестирования ученика

Для определения логики работы профориентационного теста была разработана соответствующая блок-схема, отображающая последовательность действий ученика при прохождении тестирования (Рисунок 8.3).

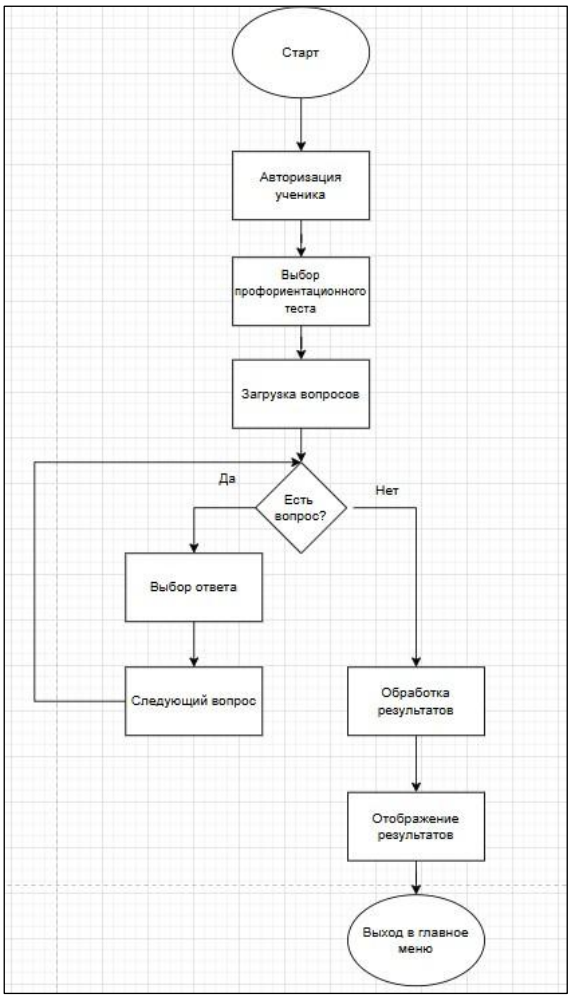

Рисунок 8.3 - Блок-схема прохождения профориентационного теста

Для отображения динамики обучения и повышения мотивации ученика была разработана блок-схема функции визуализации индивидуального прогресса (Рисунок 8.4).

Рисунок 8.4 - Блок-схема визуализации прогресса обучения

Работа функции начинается с входа ученика в личный кабинет. После выбора раздела просмотра прогресса система обращается к базе данных для получения информации о пройденных темах, результатах тестов и текущем уровне сложности.

На следующем этапе выполняется обработка данных, в ходе которой определяется общий уровень освоения материала и динамика изменений результатов обучения. Полученные данные преобразуются в наглядную форму и отображаются в интерфейсе приложения с использованием индикаторов, шкал.

Завершающим этапом является просмотр учеником своего прогресса и возврат в основное меню приложения либо переход к выполнению следующих заданий.

8.2 Разработка пользовательского интерфейса

Пользовательский интерфейс веб-приложения KiberOne разработан с целью обеспечить удобную навигацию и интуитивное взаимодействие для пользователей разных ролей: администратор, учитель, ученик. Интерфейс учитывает возрастную аудиторию (6 - 14 лет) и особенности функционала каждого пользователя.

1. Главная страница:

- центральный элемент - логотип KiberOne (Рисунок 8.5);
- три кнопки для выбора роли: Администратор, Учитель, Ученик;
- простая и наглядная структура, чтобы пользователь сразу понимал, куда перейти.

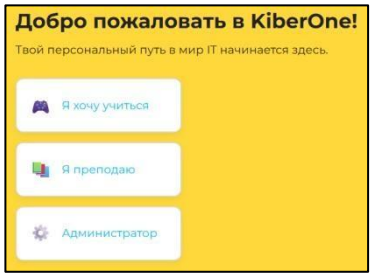

Рисунок 8.5 – Выбор роли на сайте

2. Форма входа:

- для Администратора и Учителя: поля логин и пароль, кнопка подтверждения входа;
- для Ученика: поля имя и фамилия, выбор аватара, кнопка перехода в кабинет (Рисунок 8.6);
- при некорректном вводе отображается сообщение об ошибке.

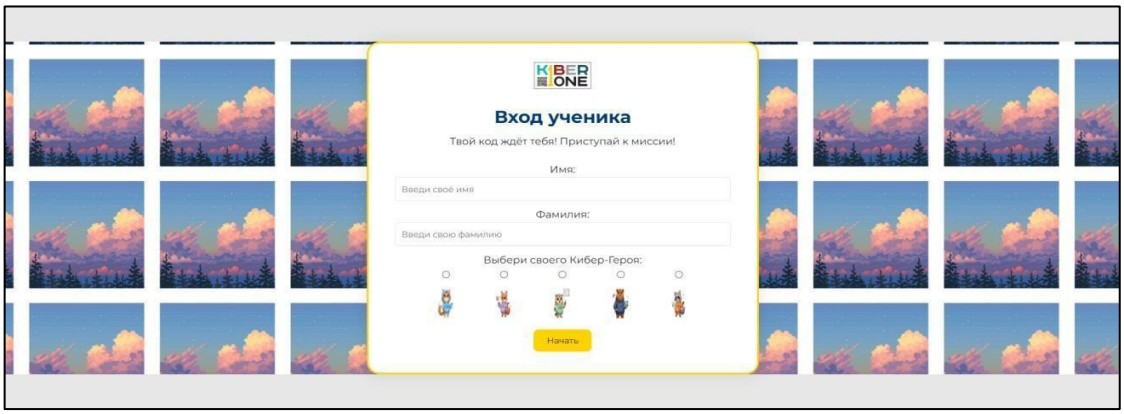

Рисунок 8.6 – Вход для Учеников

3. Кабинет администратора:

- панель с меню: Пользователи, Тесты, Результаты;
- списки пользователей с возможностью добавления, редактирования и удаления (Рисунок 8.7);
- просмотр и управление тестами, результатами учеников.

Рисунок 8.7 – Вывод список пользователей

4. Кабинет учителя:

– меню: Мои ученики, создать тест, Результаты тестов, добавить ученика (Рисунок 8.8);
– список учеников с отображением прогресса;
– форма создания и редактирования тестов: добавление вопросов, вариантов ответов, уровня сложности.

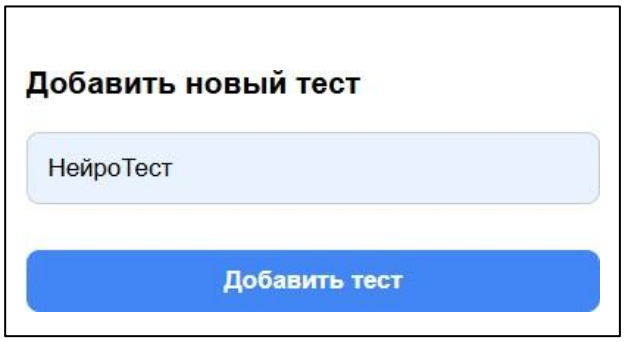

Рисунок 8.8 – Добавление теста в кабинете Учителя

5. Кабинет ученика:

- отображение аватара и имени ученика;
- тренажер «Собери клавиатуру» – три уровня сложности;
- «Стамин онлайн» – тренажер скорости печати;
- адаптивный тест по ПК – задания под уровень знаний ученика (Рисунок 8.9);
- возможность смены аватара;
- прогресс по блокам отображается визуально (полосой выполнения).

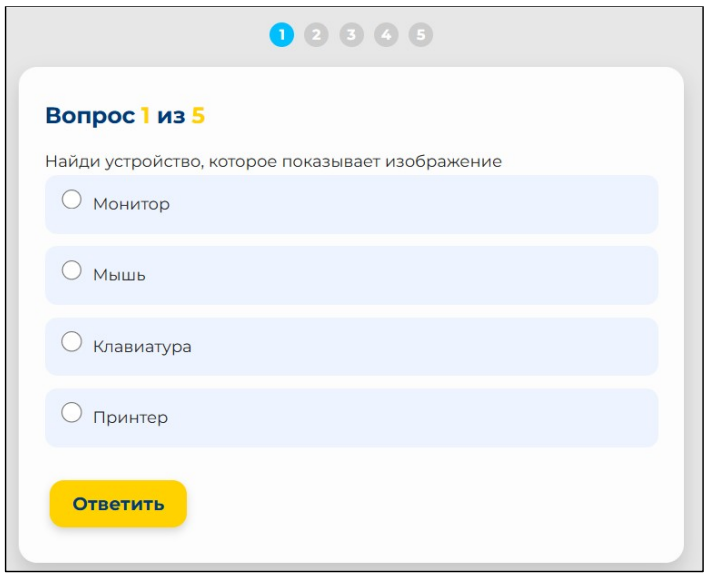

Рисунок 8.9 – Адаптивный тест в кабинете ученика

Была реализована функция анализа результатов тестирования учеников, предназначенная для использования в кабинете учителя.

Основной задачей данной функции является предоставление преподавателю наглядной и структурированной информации о ходе обучения учеников. Функция обеспечивает получение данных о прохождении тестов, количестве правильных и неправильных ответов, а также уровне сложности заданий, достигнутом каждым учеником (Рисунок 8.10).

Рисунок 8.10 - Пример отображения результатов тестирования в кабинете учителя

Основной целью профориентационного теста является выявление интересов, склонностей и предпочтений ученика в области информационных технологий. Тест позволяет помочь обучающемуся лучше понять, какие направления программирования или IT-сферы могут быть для него наиболее подходящими, что положительно влияет на мотивацию и осознанность обучения (Рисунок 8.11).

Ученик получает краткое описание рекомендуемого направления обучения, что способствует формированию интереса к дальнейшему изучению программирования.

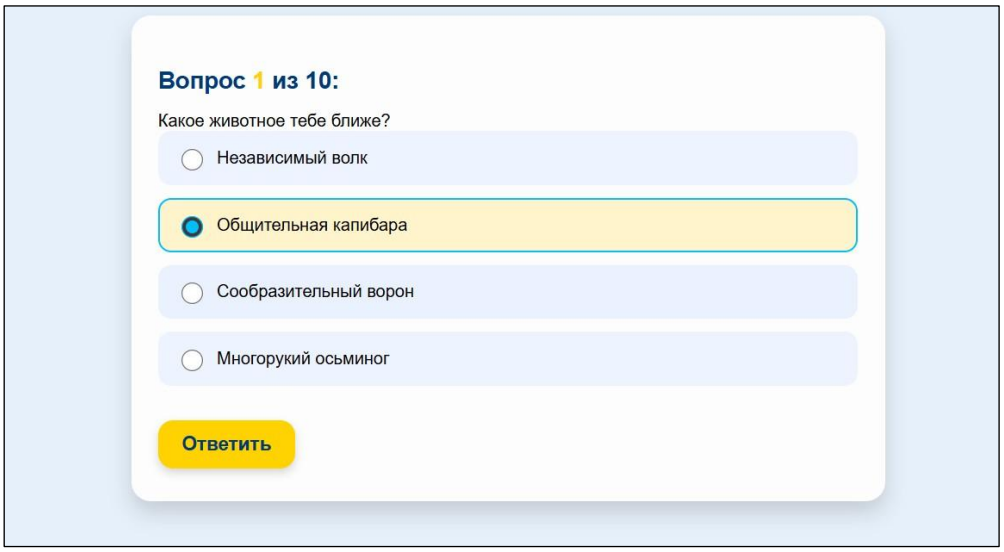

Рисунок 8.11 - Пример прохождения профориентационного теста в кабинете ученика

Для повышения наглядности и мотивации обучающихся была реализована функция визуализации индивидуального прогресса ученика. Данный функционал предназначен для отображения динамики обучения и текущих достижений пользователя в процессе освоения учебного материала.

На этапе реализации система осуществляет получение данных о результатах обучения из базы данных, после чего выполняется их обработка и представление в наглядной форме. Прогресс ученика отображается с использованием индикаторов, шкал или списков, позволяющих быстро оценить уровень освоения материала (Рисунок 8.12).

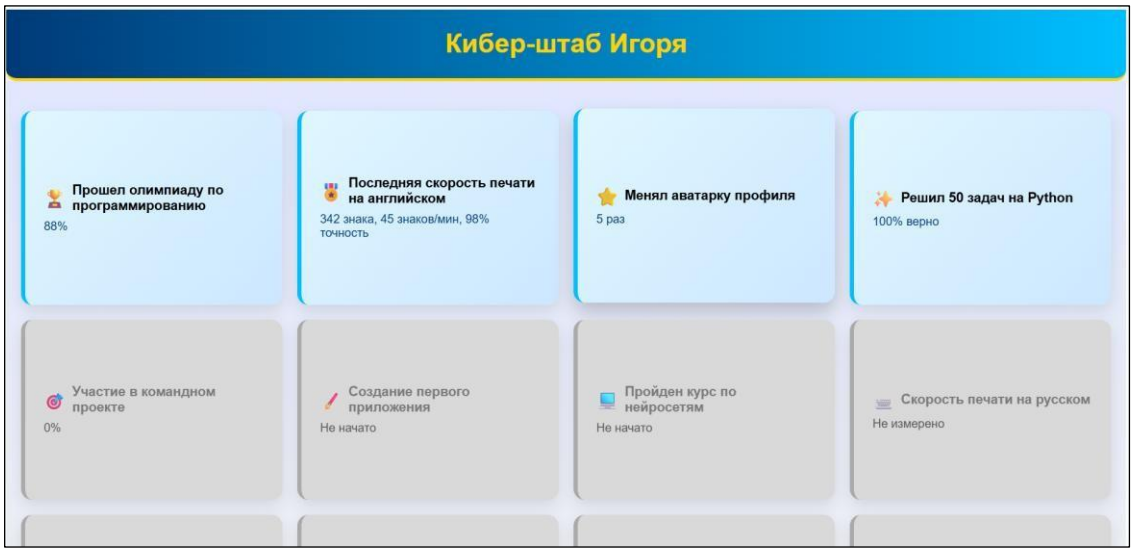

Рисунок 8.12 - Пример визуализации индивидуального прогресса ученика

Заключение

В ходе выполнения курсового проекта (учебной практики) был разработан программный модуль для автоматизации учета заявок в сервисном центре ООО «БытСервис». Данная разработка позволила перевести бизнес-процессы организации с разрозненного табличного учета на современную реляционную систему управления данными.

В процессе выполнения работы были достигнуты следующие ключевые результаты:

1. Реорганизация структуры данных: на основе анализа существующих журналов учета была спроектирована и реализована нормализованная база данных, состоящая из 6 взаимосвязанных таблиц (Roles, Users, Clients, Requests, Statuses, Comments). Это позволило устранить избыточность данных (например, ФИО клиента теперь хранится в единственном экземпляре в таблице Clients) и обеспечить целостность информации при удалении или редактировании записей.

2. Разработка многопользовательского интерфейса: средствами языка Python и библиотеки Tkinter был создан графический интерфейс, адаптированный под конкретные бизнес-роли организации:

- Заказчик получил возможность удаленной подачи и редактирования своих заявок.
- Оператор - инструмент для быстрой регистрации и поиска по базе.
- Мастер - личный кабинет для фиксации хода работ и расхода запчастей.
- Менеджер по качеству - функционал для оперативного вмешательства в проблемные заказы и продления сроков.

3. Внедрение инновационных решений по ТЗ: в соответствии с требованиями заказчика в программный модуль была интегрирована система оценки качества. Реализованный алгоритм автоматической генерации QR-кода при переводе заявки в статус «Готова к выдаче» обеспечивает прямую связь клиента с

администрацией через Google-форму, что значительно повышает прозрачность сервиса.

4. Автоматизация отчетности: разработанный модуль статистики позволяет руководству ООО «БытСервис» в режиме реального времени получать данные о количестве выполненных заявок и среднем времени ремонта, что ранее требовало ручного подсчета и занимало значительное время.

Приложение А

Таблицы «users»

Таблица «reviews»

Таблица «administrator»

Таблица «categories»

Таблица «exchanges»

Таблица «items»

Таблица «rentals»

Приложение Б

App.py

```
from flask import Flask, render_template, request, redirect, url_for, flash
from config import Config
from models import db, User, Item, Category, Rental, Review, Exchange, Administrator
from flask_login import LoginManager, login_user, current_user, logout_user, login_required
from decorators import admin_required, renter_required, user_required
from forms import (
    RegistrationForm, LoginForm, AddItemForm,
    EditCategoryForm, AddCategoryForm, AddReviewForm,
    EditUserForm, EditRentalForm, EditExchangeForm, EditItemForm
)

app = Flask(__name__)
app.config.from_object(Config)
db.init_app(app)

login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = 'login'

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/приветствие')
@login_required
```

```
def welcome():
    return render_template('welcome.html')

@app.route('/категории')
@login_required
def categories():
    categories = Category.query.all()
    return render_template('categories.html', categories=categories)

@app.route('/товары')
@login_required
def items():
    category_id = request.args.get('category_id')
    if category_id:
        items = Item.query.filter_by(categoryid=category_id).all()
    else:
        items = Item.query.all()
    categories = Category.query.all()
    return          render_template('items.html',          items=items,
categories=categories, selected_category=category_id)

@app.route('/товар/<int:item_id>')
@login_required
def item_detail(item_id):
    item = Item.query.get_or_404(item_id)
    reviews = Review.query.filter_by(itemid=item_id).all()
    return        render_template('item_detail.html',        item=item,
reviews=reviews)

@app.route('/регистрация', methods=['GET', 'POST'])
def register():
    if current_user.is_authenticated:
        return redirect(url_for('welcome'))
    form = RegistrationForm()
    if form.validate_on_submit():
```

```
        new_user = User(name=form.name.data,
lastname=form.lastname.data, email=form.email.data,
password=form.password.data, role=form.role.data)
        db.session.add(new_user)
        db.session.commit()
        flash('Ваш аккаунт был создан! Теперь вы можете войти',
'success')
        return redirect(url_for('login'))
    return render_template('register.html', form=form)

@app.route('/вход', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('welcome'))
    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(email=form.email.data).first()
        if user and user.password == form.password.data:
            login_user(user, remember=form.remember.data)
            next_page = request.args.get('next')
            if not next_page or url_parse(next_page).netloc != '':
                next_page = url_for('welcome')
            if user.role == 'пользователь':
                return redirect(url_for('user_dashboard'))
            elif user.role == 'арендодатель':
                return redirect(url_for('renter_dashboard'))
            elif user.role == 'администратор':
                return redirect(url_for('admin_dashboard'))
        else:
            flash('Неудачный вход. Пожалуйста, проверьте email и
пароль', 'danger')
    return render_template('login.html', form=form)

@app.route('/выход')
@login_required
```

```
def logout():
    logout_user()
    return redirect(url_for('welcome'))

@app.route('/редактировать_товар/<int:item_id>',       methods=['GET',
'POST'])
@login_required
@renter_required
def edit_item(item_id):
    item = Item.query.get_or_404(item_id)
    form = EditItemForm(obj=item)
    if form.validate_on_submit():
        item.name = form.name.data
        item.description = form.description.data
        item.condition = form.condition.data
        item.categoryid = form.category_id.data
        db.session.commit()
        return redirect(url_for('item_detail', item_id=item.itemid))
    categories = Category.query.all()
    return        render_template('edit_item.html',        form=form,
categories=categories)

@app.route('/админ')
@login_required
@admin_required
def admin():
    items = Item.query.all()
    rentals = Rental.query.all()
    reviews = Review.query.all()
    exchanges = Exchange.query.all()
    categories = Category.query.all()
    return render_template('admin.html', items=items, rentals=rentals,
reviews=reviews, exchanges=exchanges, categories=categories)

@app.route('/добавить_товар', methods=['GET', 'POST'])
```

```
@login_required
@renter_required
def add_item():
    form = AddItemForm()
    if form.validate_on_submit():
        name = form.name.data
        description = form.description.data
        condition = form.condition.data
        user_id = current_user.userid
        category_id = form.category_id.data
        new_item     =     Item(name=name,     description=description,
condition=condition, userid=user_id, categoryid=category_id)
        db.session.add(new_item)
        db.session.commit()
        return redirect(url_for('renter_dashboard'))
    categories = Category.query.all()
    return         render_template('add_item.html',         form=form,
categories=categories)

@app.route('/добавить_обмен', methods=['GET', 'POST'])
@login_required
@renter_required
def add_exchange():
    form = AddExchangeForm()
    if form.validate_on_submit():
        exchange_date = form.exchange_date.data
        status = form.status.data
        user_id = current_user.userid
        new_exchange      =      Exchange(exchangedate=exchange_date,
status=status, userid=user_id)
        db.session.add(new_exchange)
        db.session.commit()
        return redirect(url_for('renter_dashboard'))
    return render_template('add_exchange.html', form=form)
```

```
@app.route('/добавить_аренду', methods=['GET', 'POST'])
@login_required
@renter_required
def add_rental():
    if request.method == 'POST':
        start_date = request.form['start_date']
        end_date = request.form['end_date']
        status = request.form['status']
        item_id = request.form['item_id']
        user_id = current_user.userid
        new_rental  =  Rental(startdate=start_date,  enddate=end_date,
status=status, itemid=item_id, userid=user_id)
        db.session.add(new_rental)
        db.session.commit()
        return redirect(url_for('renter_dashboard'))
    items = Item.query.filter_by(userid=current_user.userid).all()
    return render_template('add_rental.html', items=items)

@app.route('/добавить_пользователя', methods=['GET', 'POST'])
@login_required
@admin_required
def add_user():
    form = RegistrationForm()
    if form.validate_on_submit():
        new_user              =              User(name=form.name.data,
lastname=form.lastname.data,                    email=form.email.data,
password=form.password.data, role=form.role.data)
        db.session.add(new_user)
        db.session.commit()
        flash('Пользователь был успешно добавлен!', 'success')
        return redirect(url_for('admin_dashboard', table='users'))
    return render_template('add_user.html', form=form)

@app.route('/добавить_категорию', methods=['GET', 'POST'])
@login_required
```

```
@admin_required
def add_category():
    form = AddCategoryForm()
    if form.validate_on_submit():
        name = form.name.data
        description = form.description.data
        new_category = Category(name=name, description=description)
        db.session.add(new_category)
        db.session.commit()
        return redirect(url_for('admin_dashboard'))
    return render_template('add_category.html', form=form)

@app.route('/редактировать_категорию/<int:category_id>',
methods=['GET', 'POST'])
@login_required
@admin_required
def edit_category(category_id):
    category = Category.query.get_or_404(category_id)
    form = EditCategoryForm(obj=category)
    if form.validate_on_submit():
        category.name = form.name.data
        category.description = form.description.data
        db.session.commit()
        return redirect(url_for('admin_dashboard'))
    return render_template('edit_category.html', form=form)

@app.route('/редактировать_пользователя/<int:user_id>',
methods=['GET', 'POST'])
@login_required
@admin_required
def edit_user(user_id):
    user = User.query.get_or_404(user_id)
    form = EditUserForm(obj=user)
    if form.validate_on_submit():
        form.populate_obj(user)
```

```
        db.session.commit()
        return redirect(url_for('admin_dashboard', table='users'))
    return render_template('edit_user.html', form=form)

@app.route('/редактировать_аренду/<int:rental_id>',    methods=['GET',
'POST'])
@login_required
@admin_required
def edit_rental(rental_id):
    rental = Rental.query.get_or_404(rental_id)
    form = EditRentalForm(obj=rental)
    if form.validate_on_submit():
        form.populate_obj(rental)
        db.session.commit()
        return redirect(url_for('admin_dashboard', table='rentals'))
    return render_template('edit_rental.html', form=form)

@app.route('/редактировать_обмен/<int:exchange_id>',   methods=['GET',
'POST'])
@login_required
@admin_required
def edit_exchange(exchange_id):
    exchange = Exchange.query.get_or_404(exchange_id)
    form = EditExchangeForm(obj=exchange)
    if form.validate_on_submit():
        form.populate_obj(exchange)
        db.session.commit()
        return redirect(url_for('admin_dashboard', table='exchanges'))
    return render_template('edit_exchange.html', form=form)

@app.route('/удалить_пользователя/<int:user_id>', methods=['POST'])
@login_required
@admin_required
def delete_user(user_id):
    user = User.query.get_or_404(user_id)
```

```
    db.session.delete(user)
    db.session.commit()
    return redirect(url_for('admin_dashboard', table='users'))

@app.route('/удалить_товар/<int:item_id>', methods=['POST'])
@login_required
@renter_required
def delete_item(item_id):
    item = Item.query.get_or_404(item_id)
    db.session.delete(item)
    db.session.commit()
    return redirect(url_for('renter_dashboard'))

@app.route('/удалить_категорию/<int:category_id>', methods=['POST'])
@login_required
@admin_required
def delete_category(category_id):
    category = Category.query.get_or_404(category_id)
    db.session.delete(category)
    db.session.commit()
    return redirect(url_for('admin_dashboard', table='categories'))

@app.route('/удалить_аренду/<int:rental_id>', methods=['POST'])
@login_required
@admin_required
def delete_rental(rental_id):
    rental = Rental.query.get_or_404(rental_id)
    db.session.delete(rental)
    db.session.commit()
    return redirect(url_for('admin_dashboard', table='rentals'))

@app.route('/удалить_обмен/<int:exchange_id>', methods=['POST'])
@login_required
@admin_required
def delete_exchange(exchange_id):
```

```
    exchange = Exchange.query.get_or_404(exchange_id)
    db.session.delete(exchange)
    db.session.commit()
    return redirect(url_for('admin_dashboard', table='exchanges'))

@app.route('/admin_dashboard')
@app.route('/admin_dashboard/<table>')
@login_required
@admin_required
def admin_dashboard(table=None):
    users = User.query.all()
    items = Item.query.all()
    rentals = Rental.query.all()
    reviews = Review.query.all()
    exchanges = Exchange.query.all()
    categories = Category.query.all()
    return    render_template('admin_dashboard.html',    users=users,
items=items,  rentals=rentals,  reviews=reviews,  exchanges=exchanges,
categories=categories, table=table)

@app.route('/renter_dashboard')
@login_required
@renter_required
def renter_dashboard():
    items = Item.query.filter_by(userid=current_user.userid).all()
    reviews  =  Review.query.filter(Review.itemid.in_([item.itemid  for
item in items])).all()
    return    render_template('renter_dashboard.html',    items=items,
reviews=reviews)

@app.route('/user_dashboard')
@app.route('/user_dashboard/<int:category_id>')
@login_required
@user_required
def user_dashboard(category_id=None):
```

```
    categories = Category.query.all()
    if category_id:
        items = Item.query.filter_by(categoryid=category_id).all()
    else:
        items = Item.query.all()
    return     render_template('user_dashboard.html',     items=items,
categories=categories, selected_category_id=category_id)

@app.route('/добавить_отзыв/<int:item_id>', methods=['GET', 'POST'])
@login_required
@user_required
def add_review(item_id):
    form = AddReviewForm()
    if form.validate_on_submit():
        text = form.text.data
        rating = form.rating.data
        user_id = current_user.userid
        new_review = Review(text=text, rating=rating, userid=user_id,
itemid=item_id)
        db.session.add(new_review)
        db.session.commit()
        return redirect(url_for('item_detail', item_id=item_id))
    return render_template('add_review.html', form=form)

if __name__ == '__main__':
    app.run(debug=True)
```

Config.py

```
import os

class Config:
    SQLALCHEMY_DATABASE_URI                                          =
'postgresql://381:ghjk381@194.164.33.111:5432/381_Udilova_UP'
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SECRET_KEY = os.urandom(24)
```

Decorators.py

```
# decorators.py
from functools import wraps
from flask import redirect, url_for, flash
from flask_login import current_user

def admin_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if current_user.role != 'администратор':
            flash('У вас нет прав доступа к этой странице.', 'danger')
            return redirect(url_for('index'))
        return f(*args, **kwargs)
    return decorated_function

def renter_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if current_user.role != 'арендодатель':
            flash('У вас нет прав доступа к этой странице.', 'danger')
            return redirect(url_for('index'))
        return f(*args, **kwargs)
    return decorated_function

def user_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if current_user.role != 'пользователь':
            flash('У вас нет прав доступа к этой странице.', 'danger')
            return redirect(url_for('index'))
        return f(*args, **kwargs)
    return decorated_function
```

Forms.py

```
# forms.py
from flask_wtf import FlaskForm
from wtforms import DateField
```

```
from wtforms import StringField, TextAreaField, SubmitField, SelectField, IntegerField, PasswordField, BooleanField
from wtforms.validators import DataRequired, Length, EqualTo, NumberRange, Email, ValidationError
from models import Category, User, Item
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField, BooleanField
from wtforms.validators import DataRequired, Email, Length, EqualTo


class RegistrationForm(FlaskForm):
    name = StringField('Имя', validators=[DataRequired(), Length(min=2, max=255)])
    lastname = StringField('Фамилия', validators=[DataRequired(), Length(min=2, max=255)])
    email = StringField('Email', validators=[DataRequired(), Email()])
    password = PasswordField('Пароль', validators=[DataRequired(), Length(min=6)])
    role = SelectField('Роль', choices=[('пользователь', 'Пользователь'), ('арендодатель', 'Арендодатель'), ('администратор', 'Администратор')], validators=[DataRequired()])
    submit = SubmitField('Зарегистрироваться')

    def validate_email(self, email):
        user = User.query.filter_by(email=email.data).first()
        if user:
            raise ValidationError('Этот email уже зарегистрирован. Пожалуйста, выберите другой.')


class LoginForm(FlaskForm):
    email = StringField('Email', validators=[DataRequired(), Email()])
    password = PasswordField('Пароль', validators=[DataRequired(), Length(min=6)])
```

```
    remember = BooleanField('Запомнить меня')
    submit = SubmitField('Войти')

class AddItemForm(FlaskForm):
    name    =    StringField('Название',    validators=[DataRequired(),
Length(min=2, max=255)])
    description              =              TextAreaField('Описание',
validators=[DataRequired()])
    condition = StringField('Состояние', validators=[DataRequired()])
    category_id      =      SelectField('Категория',      coerce=int,
validators=[DataRequired()])
    submit = SubmitField('Добавить товар')

    def __init__(self, *args, **kwargs):
        super(AddItemForm, self).__init__(*args, **kwargs)
        self.category_id.choices       =       [(category.categoryid,
category.name) for category in Category.query.all()]

class EditUserForm(FlaskForm):
    name     =     StringField('Имя',     validators=[DataRequired(),
Length(min=2, max=255)])
    lastname   =   StringField('Фамилия',   validators=[DataRequired(),
Length(min=2, max=255)])
    email = StringField('Email', validators=[DataRequired(), Email()])
    role     =     SelectField('Роль',     choices=[('пользователь',
'Пользователь'),  ('арендодатель',  'Арендодатель'),  ('администратор',
'Администратор')], validators=[DataRequired()])
    submit = SubmitField('Сохранить изменения')

class EditItemForm(FlaskForm):
    name    =    StringField('Название',    validators=[DataRequired(),
Length(min=2, max=255)])
    description              =              TextAreaField('Описание',
validators=[DataRequired()])
    condition = StringField('Состояние', validators=[DataRequired()])
```

```
    category_id      =      SelectField('Категория',      coerce=int,
validators=[DataRequired()])
    submit = SubmitField('Сохранить изменения')

    def __init__(self, *args, **kwargs):
        super(EditItemForm, self).__init__(*args, **kwargs)
        self.category_id.choices       =       [(category.categoryid,
category.name) for category in Category.query.all()]

class AddCategoryForm(FlaskForm):
    name    =    StringField('Название',    validators=[DataRequired(),
Length(min=2, max=255)])
    description = TextAreaField('Описание')
    submit = SubmitField('Добавить категорию')

class AddExchangeForm(FlaskForm):
    exchange_date         =         DateField('Дата         обмена',
validators=[DataRequired()])
    status = StringField('Статус', validators=[DataRequired()])
    submit = SubmitField('Добавить обмен')

class EditCategoryForm(FlaskForm):
    name    =    StringField('Название',    validators=[DataRequired(),
Length(min=2, max=255)])
    description = TextAreaField('Описание')
    submit = SubmitField('Сохранить изменения')

class EditRentalForm(FlaskForm):
    start_date = DateField('Дата начала', validators=[DataRequired()])
    end_date          =          DateField('Дата          окончания',
validators=[DataRequired()])
    status = StringField('Статус', validators=[DataRequired()])
    item_id         =         SelectField('Товар',         coerce=int,
validators=[DataRequired()])
    submit = SubmitField('Сохранить изменения')
```

```
    def __init__(self, *args, **kwargs):
        super(EditRentalForm, self).__init__(*args, **kwargs)
        self.item_id.choices = [(item.itemid, item.name) for item in
Item.query.all()]

    def __init__(self, *args, **kwargs):
        super(EditRentalForm, self).__init__(*args, **kwargs)
        self.item_id.choices = [(item.itemid, item.name) for item in
Item.query.all()]

class AddReviewForm(FlaskForm):
    text = TextAreaField('Текст отзыва', validators=[DataRequired()])
    rating = IntegerField('Рейтинг', validators=[DataRequired(),
NumberRange(min=1, max=5)])
    submit = SubmitField('Добавить отзыв')

class EditExchangeForm(FlaskForm):
    exchange_date = DateField('Дата обмена',
validators=[DataRequired()])
    status = StringField('Статус', validators=[DataRequired()])
    submit = SubmitField('Сохранить изменения')
```

Models.py

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import Column, Integer, String, Text, ForeignKey, Date
from sqlalchemy.orm import relationship
from flask_login import UserMixin #Add this line for UserMixin


db = SQLAlchemy()

class User(db.Model, UserMixin):  #Inherit from UserMixin for Flask-
Login
    __tablename__ = 'users'
    userid = Column(Integer, primary_key=True)
```

```
    name = Column(String(255), nullable=False)
    lastname = Column(String(255), nullable=False)
    email = Column(String(255), unique=True, nullable=False)
    password = Column(String(255), nullable=False)
    role = Column(String(50), nullable=False)

    def get_id(self):
        return str(self.userid) #Changed to string


class Item(db.Model):
    __tablename__ = 'items'
    itemid = Column(Integer, primary_key=True)
    name = Column(String(255), nullable=False)
    description = Column(Text, nullable=True)
    condition = Column(String(50), nullable=False)
    userid     =     Column(Integer,     ForeignKey('users.userid'),
nullable=False)
    categoryid = Column(Integer, ForeignKey('categories.categoryid'),
nullable=False)


class Category(db.Model):
    __tablename__ = 'categories'
    categoryid = Column(Integer, primary_key=True)
    name = Column(String(255), nullable=False)
    description = Column(Text, nullable=True)


class Rental(db.Model):
    __tablename__ = 'rentals'
    rentalid = Column(Integer, primary_key=True)
    startdate = Column(Date, nullable=False)
    enddate = Column(Date, nullable=False)
    status = Column(String(50), nullable=False)
```

```
    itemid      =      Column(Integer,      ForeignKey('items.itemid'),
nullable=False)
    userid      =      Column(Integer,      ForeignKey('users.userid'),
nullable=False)


class Review(db.Model):
    __tablename__ = 'reviews'
    reviewid = Column(Integer, primary_key=True)
    text = Column(Text, nullable=False)
    rating = Column(Integer, nullable=False)
    userid      =      Column(Integer,      ForeignKey('users.userid'),
nullable=False)
    itemid      =      Column(Integer,      ForeignKey('items.itemid'),
nullable=False)


class Exchange(db.Model):
    __tablename__ = 'exchanges'
    exchangeid = Column(Integer, primary_key=True)
    exchangedate = Column(Date, nullable=False)
    status = Column(String(50), nullable=False)
    userid      =      Column(Integer,      ForeignKey('users.userid'),
nullable=False)


class Administrator(db.Model):
    __tablename__ = 'administrator'
    adminid = Column(Integer, primary_key=True)
    userid      =      Column(Integer,      ForeignKey('users.userid'),
nullable=False)
```